1. The `is` Operator (Arithmetic Evaluation)

In Prolog, arithmetic expressions such as `3 + 5` are treated as **terms**, not automatically evaluated.

```
?- 3 + 5 = 8.
No
```

This happens because `3 + 5` is a compound term, while `8` is a number. To evaluate arithmetic expressions, use the built-in operator `is`:

```
?- X is 3 + 5, X = 8.
X = 8
Yes

?- 8 is 3 + 5.
Yes
```

Rules for Using `is`

- The **right-hand side** must be a valid arithmetic expression with all variables instantiated.
- The **left-hand side** must be a variable or a number representing the result.
- Example of incorrect usage:

```
?- 3 + 5 is 8.
No
```

- If one operand is a float, the result will be a float:

```
?- 1 is 0.5 + 0.5.    % fails because 1 \= 1.0
```

- Use `is` only for **evaluation**, not for comparison.

2. Arithmetic Functions and Relations

(a) Functions (used with `is`)

Arithmetic functions can appear on the right side of `is`:

```
2 + (-3.2 * X - max(17, X)) / 2 ** 5
```

Common arithmetic functions:

- `+`, `-`, `*`, `/`, `**` — Basic math and power.
- `max(A,B)`, `min(A,B)` — Maximum, Minimum.
- `abs(X)` — Absolute value.
- `sqrt(X)` — Square root.
- `sin(X)` — Sine.
- `//` — Integer division.
- `mod` — Modulo (remainder).
- `round(X)` — Round float to nearest integer.
- `float(X)` — Convert integer to float.

Operator precedence follows normal mathematical rules:

$$2 * 3 + 4 \text{ is equivalent to } (2 * 3) + 4$$

(b) Relations (used for comparison)

Arithmetic relations compare **evaluated expressions**. The `is` operator is not required.
Common arithmetic relations:

- `X > Y` — greater than
- `X < Y` — less than
- `X =< Y` — less than or equal
- `X >= Y` — greater than or equal
- `X =:= Y` — arithmetically equal
- `X =\= Y` — arithmetically not equal

Difference between = and =:=:

```
?- 2 ** 3 =:= 3 + 5.
Yes

?- 2 ** 3 = 3 + 5.
No
```

The operator `=` checks for **syntactic equality**, whereas `=:=` checks for **arithmetic equality**.

```
?- 7 =:= 3.5 + 3.5.
Yes

?- 7 is 3.5 + 3.5.
No
```

3.Using `fail/0` in Prolog

`fail/0` is a built-in Prolog predicate that always fails. It is often used to force **backtracking**, allowing Prolog to try all possible solutions.

In this example, `fail` is used to print the names of all students one by one.

```
student(anu).
student(bala).
student(chris).

print_students :-
    student(Name),
    write(Name), nl,
    fail.   % force backtracking to find next student
print_students.
```

Query:

```
?- print_students.
```

Output:

```
anu
bala
chris
true.
```

Explanation:

- The first rule finds a student and prints their name.
- The `fail` predicate forces Prolog to backtrack and look for the next student.
- Once all students have been printed, the first rule fails completely.
- The second rule `print`$_s$*tudents.thensucceedsquietly, endingtheloop.*

4. Using `between/3` in Prolog

`between/3` is a built-in Prolog predicate used to generate or test integers within a given range.

- The syntax is `between(Low, High, X)`.
- It succeeds when `X` takes on all integer values from `Low` to `High`, one by one.
- On backtracking, it generates the next value.

```
print_numbers(Low, High) :-
    between(Low, High, X),
    write(X), nl,
    fail.
print_numbers(_, _).
```

Query:

```
?- print_numbers(1, 5).
```

Output:

```
1
2
3
4
5
true.
```

Explanation:

- The first clause uses `between/3` to generate numbers from 1 to 5.
- Each time, it writes the current number and then hits `fail`, forcing Prolog to backtrack.
- Backtracking causes `between/3` to produce the next number.
- After the last number, `between/3` fails and Prolog moves to the second clause, which simply succeeds.

5. Key Takeaways

- Use `is` to **evaluate** arithmetic expressions.
- Use `=:=` or other arithmetic relations to **compare** evaluated values.
- Be aware of **float vs integer** behavior.
- Functions like `max/2`, `sqrt/1`, and `mod` can be used within expressions.

Exercise 3.1

Write a Prolog predicate `distance/3` to calculate the distance between two points in the 2-dimensional plane. Points are given as pairs of coordinates.

Examples:

```
?- distance((0,0), (3,4), X).
X = 5.0
Yes

?- distance((-2.5,1), (3.5,-4), X).
X = 7.810249675906654
Yes
```

Exercise 3.2

Write a Prolog program to print out a square of `n × n` given characters on the screen. Call your predicate `square/2`. The first argument should be a (positive) integer, the second argument the character (any Prolog term) to be printed.

Example:

```
?- square(5, *).
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
Yes
```

Exercise 3.3

Write a Prolog predicate `fibonacci/2` to compute the nth Fibonacci number. The Fibonacci sequence is defined as follows:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ for } n \geq 2$$

Examples:

```
?- fibonacci(1, X).
X = 1
Yes

?- fibonacci(3, X).
X = 2
Yes

?- fibonacci(7, X).
X = 13
Yes
```

Exercise 3.4

This exercise assumes you have already solved the previous one. The straightforward implementation of Fibonacci is not very efficient and will run out of memory for larger numbers.

Examples:

```
?- fibonacci(10, X).
X = 55
Yes

?- fibonacci(20, X).
X = 6765
Yes

?- fibonacci(50, X).
ERROR: Out of local stack
```

Briefly explain what the source of this problem is. Then write a predicate `fastfibo/2` that can compute any of the first 100 Fibonacci numbers efficiently.

Examples:

```
?- fastfibo(50, X).
X = 12586269025
Yes

?- fastfibo(100, X).
X = 354224848179261915075
Yes
```

What is the 42nd Fibonacci number?

Exercise 3.5

Write a predicate `element_at/3` that, given a list and a natural number n, will return the nth element of that list.

Examples:

```
?- element_at([tiger, dog, teddy_bear, horse, cow], 3, X).
X = teddy_bear
Yes

?- element_at([a, b, c, d], 27, X).
No
```

Exercise 3.6

Write a predicate `mean/2` to compute the arithmetic mean of a given list of numbers.

Example:

```
?- mean([1, 2, 3, 4], X).
X = 2.5
Yes
```

Exercise 3.7

Write a predicate `minimum/2` to find the smallest number within a given list of numbers.

Example:

```
?- minimum([4, 6, 8, 3, 5, 7], Result).
Result = 3
Yes
```

What does your predicate do when given the empty list as input? Is that the correct answer? Why?

Exercise 3.8

Write a predicate `range/3` to generate all integers between a given lower and a given upper bound. The lower bound should be given as the rst argument, the upper bound as the second. The result should be a list of integers, which is returned in the third argument position. If the upper bound speci ed is lower than the given lower bound, the empty list should be returned. Examples:

Examples:

```
?- range(3, 11, X).
X = [3, 4, 5, 6, 7, 8, 9, 10, 11]
Yes

?- range(7, 4, X).
X = []
Yes
```

Exercise 3.9

This exercise demonstrates a simple Prolog database. Copy the following facts:

```
born(jan, date(20,3,1977)).
born(jeroen, date(2,2,1992)).
born(joris, date(17,3,1995)).
born(jelle, date(1,1,2004)).
born(jesus, date(24,12,0)).
born(joop, date(30,4,1989)).
born(jannecke, date(17,3,1993)).
born(jaap, date(16,11,1995)).
```

(a) Write a predicate `year/2` to retrieve all people born in a given year.

```
?- year(1995, Person).
Person = joris ;
Person = jaap ;
No
```

(b) Implement a predicate `before/2` that, when given two date-expressions, will suc ceed if the rst expression represents a date before the date represented by the second expression (you may assume that the user will only ask for well-formed dates, e.g., not for the 31st of April, and so forth). Example

```
?- before(date(31,1,1990), date(7,7,1990)).
Yes
```

(c) Implement `older/2` that succeeds if the first person is older than the second.

```
?- older(jannecke, X).
X = joris ;
X = jelle ;
X = jaap ;
No
```

Exercise 3.10

Imagine a robot that can turn right, turn left, and move forward. You must write `status/3` that returns the final position and orientation after executing a list of commands.

Examples:

```
?- status([move, move, right, move, move, move], Position, Orientation).
Position = (3,2)
Orientation = east
Yes

?- status([], Position, Orientation).
Position = (0,0)
Orientation = north
Yes

?- status([left, left, move], Position, Orientation).
Position = (0,-1)
Orientation = south
Yes
```

Exercise 3.11

Polynomials can be represented as lists of (coefficient, exponent) pairs.

$$4x^5 + 2x^3 - x + 27 \Rightarrow [(4,5),(2,3),(-1,1),(27,0)]$$

Write a predicate `poly_sum/3` to add two polynomials.

Examples:

```
?- poly_sum([(5,3), (1,2)], [(1,3)], Sum).
Sum = [(6,3), (1,2)]
Yes

?- poly_sum([(2,2), (3,1), (5,0)], [(5,3), (1,1), (10,0)], X).
X = [(4,1), (15,0), (2,2), (5,3)]
Yes
```

Exercise 3.12

Write a predicate `prime/1` to check whether a given number is prime.

Examples:

```
?- prime(17).
Yes

?- prime(18).
No
```

Exercise 3.13

Write a predicate `goldbach/2` that expresses an even integer as a sum of two prime numbers.

Examples:

```
?- goldbach(30, Solution).
Solution = 7+23
Yes

?- goldbach(17420000, Solution).
Solution = 109+17419891
Yes
```